

Zero Carbon One – Agentic AI Utility Bill Extraction Assignment

Objective

Build a full-stack application that enables users to upload utility bills and automatically extract structured data using local AI models, OCR and Agentic AI workflows.

The platform should process utility bills from different vendors and formats and return standardized data that can be used for carbon accounting, energy management and sustainability reporting.

The objective is to evaluate your ability to design and implement an AI-powered document processing system using React, Node.js and locally hosted AI models.

Business Context

Manufacturing companies receive thousands of utility bills every year including:

- Electricity Bills
- Coal Purchase Invoices
- Diesel Purchase Invoices
- Natural Gas Bills
- LPG Bills
- Water Bills
- Steam Bills
- Renewable Energy Certificates
- Fuel Transportation Invoices

These documents arrive in multiple formats and layouts.

The system should automatically identify the document type, extract relevant fields and store them in a structured format.

Technology Requirements

Frontend

- React JS

Backend

- Node.js

Database

- MongoDB

AI Components

You may use either:

Option A – Local Processing (Preferred)

- Ollama
- LangChain
- Tesseract
- PaddleOCR
- EasyOCR
- Open-source models

Option B – Cloud APIs (Allowed for Assignment)

- OpenAI API
- Azure Document Intelligence
- AWS Textract
- Google Vision OCR

You should clearly document their chosen approach.

Functional Requirements

1. Document Upload

Support:

- PDF

- PNG
- JPG
- JPEG

Allow:

- Single file upload
- Multiple file upload

Display:

- Upload progress
- Processing status
- Extraction result

2. OCR Processing

Implement a hybrid OCR pipeline:

Step 1

Attempt direct text extraction from PDFs.

Step 2

If extraction quality is poor:

- Convert pages to images
- Execute OCR

Step 3

Combine extracted content.

3. Agentic AI Extraction Workflow

Develop an Agentic AI workflow that:

Task 1

Identify document category:

Examples:

- Electricity Bill
- Diesel Invoice
- Coal Invoice
- Water Bill
- Gas Bill

Task 2

Identify vendor or utility provider.

Examples:

- Tata Power
- Adani Electricity
- BSES
- Reliance Energy
- IOCL
- BPCL
- HPCL

Task 3

Extract relevant structured fields.

Mandatory Extraction Fields

Electricity Bills

Extract:

- Utility Provider
- Consumer Number
- Account Number
- Billing Period
- Bill Date
- Due Date
- Units Consumed (kWh)

- Contract Demand
- Maximum Demand
- Total Bill Amount
- GST Amount

Diesel Invoices

Extract:

- Supplier Name
- Invoice Number
- Invoice Date
- Quantity (Litres)
- Rate Per Litre
- Total Amount

Coal Invoices

Extract:

- Supplier Name
- Invoice Number
- Invoice Date
- Coal Grade
- Quantity (Tonnes)
- GCV
- Moisture %
- Total Amount

Water Bills

Extract:

- Supplier Name
- Billing Period
- Consumption Volume
- Unit
- Total Amount

Standardized JSON Output

Output should be generated in a standardized format.

Example:

```
{
  "document_type": "electricity_bill",
  "vendor": "Tata Power",
  "bill_date": "2026-01-15",
  "billing_period": "Dec 2025",
  "consumption_kwh": 154230,
  "total_amount": 1540000,
  "confidence_score": 0.95
}
```

4. Agentic Validation Layer

The AI workflow should validate:

- Missing fields
- Incorrect units
- Duplicate invoices
- Suspicious values

Examples:

- Negative consumption
- Missing invoice date
- Missing quantity

Display validation warnings.

5. Human Review Screen

Allow users to:

- Review extracted values
- Edit values
- Approve extraction

Store final approved output.

6. MongoDB Storage

Store:

- Uploaded documents
- OCR output
- AI extracted output
- User corrections
- Approval status

Background Processing Queue

Implement:

Upload → Queue → OCR → AI Extraction → Validation → Storage

Using:

- BullMQ
- Redis

or equivalent.

Role Based Access Control

Admin

- View all documents
- Manage users

User

- Access own documents

Docker Deployment

Provide:

- Dockerfile
- Docker Compose

Application should run using:

`docker-compose up`

Evaluation Criteria

Area	Weightage
Agentic AI Workflow Design	25%
OCR & Extraction Accuracy	25%
Backend Architecture	20%
React Frontend	10%
MongoDB Design	10%
Error Handling & Validation	5%
Documentation	5%

Submission Requirements

1. GitHub Repository
2. Architecture Diagram
3. README

Include:

- Setup instructions
 - AI models used
 - Extraction approach
 - Design decisions
4. Docker Setup
 5. 5-Minute Demo Video

Demonstrate:

- Upload process
- OCR workflow
- Agentic extraction workflow
- Validation process
- Final structured output

What We Are Evaluating

- React JS proficiency
- Node.js architecture
- Agentic AI implementation capability
- OCR integration skills
- MongoDB design
- Enterprise software thinking
- Ability to build AI products for industrial sustainability use cases